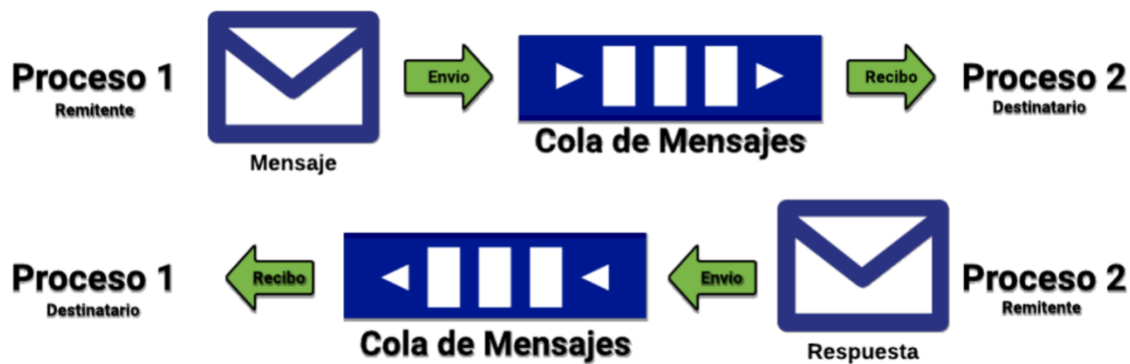


Cola de mensajes

Cola de mensajes en C bajo Linux

Las colas de mensajes, junto con los semáforos y la memoria compartida, son los recursos compartidos que pone Linux a disposición de los programas para que puedan intercambiarse información.

En C para Linux es posible hacer que dos procesos (dos programas) distintos sean capaces de enviarse mensajes y de esta forma pueden intercambiar información. El mecanismo para conseguirlo es el de una cola de mensajes. Los procesos introducen mensajes en la cola y se van almacenando en ella. Cuando un proceso extrae un mensaje de la cola, extrae el primer mensaje que se introdujo y dicho mensaje se borra de la cola.



También es posible hacer "tipos" de mensajes distintos, de forma que cada tipo de mensaje contiene una información distinta y **va identificado por un entero**. Por ejemplo:

Los "tipos" de mensajes se los llama **EVENTOS**; cada evento se identificará con un valor único.

Ejemplo:

```
typedef enum
{
    EVT_NINGUNO,           //valor 0
    EVT_CONSULTA_STOCK,    //valor 1
    EVT_RESPUESTA_STOCK,   //valor 2
    EVT_SUMA_STOCK,        //valor 3
    EVT_RESTA_STOCK        //valor 4
} Eventos;
```

La forma de conseguir una cola de mensajes en un programa es la siguiente:

- En primer lugar, se necesita conseguir una clave, de tipo `key_t`, que sea común para todos los programas que quieran compartir la cola de mensajes. Para ello existe la función `key_tftok (char *, int)`. A dicha función se le pasa un fichero que exista y sea accesible y un entero. Con ellos construye una clave que devuelve. Si todos los programas utilizan el mismo fichero y el mismo entero, obtendrán la misma clave. Es habitual, como primer parámetro, pasar algún fichero del sistema que se esté seguro de su existencia, como por ejemplo `"/bin/lis"`, que es el `"ls"` del unix. Para el entero, bastaría con poner un `#define` en algún fichero.h de forma que todos los programas que vayan a utilizar la misma cola incluyan dicho fichero y utilicen como entero el del `#define`.
- Una vez obtenida la clave, se crea la cola de mensajes. Para ello está la función `intmsgget (key_t, int)`. Con dicha función se crea la cola y devuelve un identificador para la misma. Si la cola correspondiente a la Clave `key_t` ya estuviera creada, simplemente daría el identificador de la misma (siempre y cuando los parámetros no indiquen lo contrario). El primer parámetro es la clave `key_t` obtenida anteriormente y que debería ser la misma para todos los programas. El segundo parámetro son unos *flags*. Aunque hay más posibilidades, lo imprescindible es:
 - 9 bits menos significativos, son permisos de lectura/escritura/ejecución para propietario/grupo/otros, al igual que los ficheros. Para obtener una cola con todos los permisos para todo el mundo, es necesario poner como parte de los *flags* el número 0777. Es importante el cero delante, para que el número se interprete en octal y queden los bits en su sitio (en C, cualquier número que empiece por cero, se considera octal). El de ejecución no tiene sentido y se ignora.

- IPC_CREAT. Junto con los bits anteriores, este bit indica si se debe crear la cola en caso de que no exista. Si está puesto, la cola se creará si no lo está ya y se devolverá el identificador. Si no está puesto, se intentará obtener el identificador y se obtendrá un error si no está ya creada. En resumen, los *flags* deberían ser algo así como 0777 | IPC_CREAT:

```
int creo_idColaMensajes(int clave)
{
    int idColaMensajes = msgget (creo_clave(clave), 0600 | IPC_CREAT);
    if (idColaMensajes == -1)
    {
        printf("Error al obtener identificador para cola mensajes\n");
        exit (-1);
    }
    return idColaMensajes;
}
```

- Ya es posible utilizar la cola de mensajes. Para meter un mensaje en la cola, se utiliza la función `msgsnd (int, structmsgbuf *, int, int)`. El primer parámetro entero es el identificador de la cola obtenido con `msgget()`. El segundo parámetro es el mensaje en sí. El mensaje debe ser una estructura cuyo primer campo sea un *long*. En dicho *long* se almacena el tipo de mensaje. El resto de los campos pueden ser cualquier cosa que se desee enviar (otra estructura, campos sueltos, etc.). Al pasar el mensaje como parámetro, se pasa un puntero al mensaje y se le hace un "cast" a `structmsgbuf *`. No hay ningún problema en este "cast" siempre y cuando el primer campo del mensaje sea un *long*. El tercer parámetro es el tamaño en bytes del mensaje exceptuando el *long*, es decir, el tamaño en bytes de los campos con la información. El cuarto parámetro son *flags*. Aunque hay varias opciones, la más habitual es poner un 0 o bien `IPC_NOWAIT`. En el primer caso la llamada a la función queda bloqueada hasta que se pueda enviar el mensaje. En el segundo caso, si el mensaje no se puede enviar, se vuelve inmediatamente con un error. El motivo habitual para que el mensaje no se pueda enviar es que la cola de mensajes esté llena.

```
int enviarMensaje(int idColaMensajes,
                  long rLongDest,
                  int rIntRte,
                  int rIntEvento,
                  char* rpCharMsg)
{
    mensaje msg;
```

```

msg.long_dest = rLongDest;
msg.int_rte   = rIntRte;
msg.int_evento = rIntEvento;
strcpy(msg.char_mensaje, rpCharMsg);
return msgsnd (idCola_mensajes,
               (struct msgbuf *)&msg,
               sizeof(msg.int_rte)+sizeof(msg.int_evento)+sizeof(msg.char_mensaje),
               IPC_NOWAIT);
}

```

Llamar a la función:

```

enviar_mensaje(idCola_mensajes, MSG_SUPERMERCADO, MSG_REPOSITOR, EVT_SUMA_STOCK, "SUMA UNO");

```

- Para recoger un mensaje de la cola se utiliza la función `msgrcv (int, structmsgbuf *, int, int, int)`. El primer parámetro es el identificador de la cola obtenido con `msgget()`. El segundo parámetro es un puntero a la estructura donde se desea recoger el mensaje. Puede ser, como en la función anterior, cualquier estructura cuyo primer campo sea un *long* para el tipo de mensaje. El tercer parámetro entero es el tamaño de la estructura exceptuando el *long*. El cuarto parámetro entero es el tipo de mensaje que se quiere retirar. Se puede indicar un entero positivo para un tipo concreto o un 0 para cualquier tipo de mensaje. El quinto parámetro son *flags*, que habitualmente puede ser 0 o bien `IPC_NOWAIT`. En el primer caso, la llamada a la función se queda bloqueada hasta que haya un mensaje del tipo indicado. En el segundo caso, se vuelve inmediatamente con un error si no hay mensaje de dicho tipo en la cola.

```

int recibir_mensaje(int idCola_mensajes,
                   long rLongDest,
                   mensaje* rMsg)
{
    mensaje msg;
    int res;
    res = msgrcv (idCola_mensajes,
                  (struct msgbuf *)&msg,
                  sizeof(msg.int_rte)+sizeof(msg.int_evento)+sizeof(msg.char_mensaje),
                  rLongDest, 0);
}

```

```

rMsg->long_dest      = msg.long_dest;
rMsg->int_rte         = msg.int_rte;
rMsg->int_evento      = msg.int_evento;
strcpy(rMsg->char_mensaje, msg.char_mensaje);
return res;
}

```

Llamar a la función:

```

recibir_mensaje(idCola_mensajes, MSG_SUPERMERCADO, &msg);

```

El receptor: “Recibe todos los mensajes para él.”

Todos los datos del mensaje: “Cuando se recibe un mensaje se cargan los datos del *struct* del mensaje”.

La función “**recibir_mensaje**” es bloqueante → hasta que no recibe un mensaje no avanza a la siguiente línea de código.

- Una vez terminada de usar la cola, se debe liberar. Para ello se utiliza la función `msgctl (int, int, structmsqid_ds *)`. Es una función genérica para control de la cola de mensajes con posibilidad de varios comandos. Aquí sólo se explica cómo utilizarla para destruir la cola. El primer parámetro es el identificador de la cola de mensajes, obtenido con `msgget()`. El segundo parámetro es el comando que se desea ejecutar sobre la cola, en este caso `IPC_RMID`. El tercer parámetro son datos necesarios para el comando que se quiera ejecutar. En este caso no se necesitan datos y se pasará un `NULL`.